Dynamic load balancer

Performance-Based Dynamic Load Balancer with Secure Persistent Backend

Student:	Shashank Yadav
Roll Number:	12342010
Systems:	stu31_sys1 to sys4
Date:	September 12, 2026
Load Balancer URL:	http://10.1.75.79:4201/

This report documents an extension of the secure group-chat application to a distributed backend deployment fronted by a custom, performance-aware Load Balancer written in Go. Clients interact exclusively through the Load Balancer URL; no client ever communicates directly with a backend server. The system replaces simple round-robin routing with a dynamic, EWMA-based scoring algorithm that continuously adapts to real backend load and latency.

1 System Overview

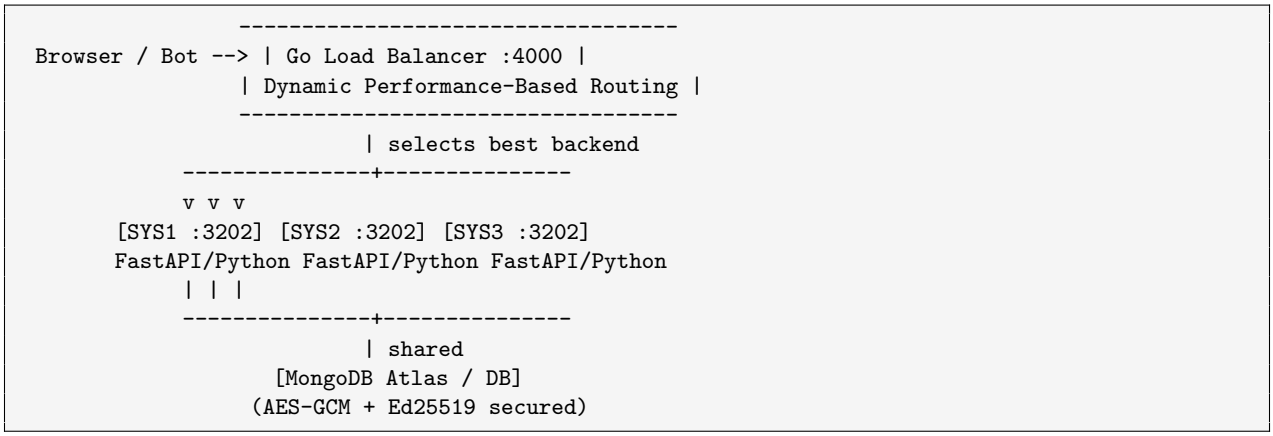

Figure 1. Three FastAPI backend replicas share a single MongoDB instance for state, while the Go load balancer continuously scores each replica and routes both HTTP and WebSocket traffic to the best-performing one.

2 Backend Architecture

2.1 Technology Stack

2.2 Required API Endpoints

`POST /message` Accepts a message from any client. Input is accepted as JSON, form data, or query parameters.

Layer	Technology
Language	Python 3.11
Web Framework	FastAPI + Uvicorn
WebSocket	Native FastAPI WebSocket
Database	MongoDB Atlas (shared, cloud)
Crypto	`cryptography` library (AES-GCM 256 + Ed25519)

```
Parameters:
  client-name -- sender username
  msg -- message text
  id -- optional deduplication ID (UUID generated if absent)
  room -- target room (defaults to "general")
  timestamp -- epoch ms (auto-generated if absent)

Response: { "status": "ok", "id": "...", "client-name": "...", "msg": "...", ... }
```

`GET /feed` Returns all stored messages, decrypted and signature-verified, in chronological order.

```
Parameters:
  room -- optional room filter
  limit -- max messages (default 2000)

Response: [ { "id", "client-name", "username", "msg", "text",
              "room", "timestamp", "verified", "tampered" }, ... ]
```

`GET /ws` (WebSocket) Real-time bidirectional chat. The Load Balancer transparently proxies the WebSocket handshake and all message frames to the selected backend.

2.3 Message Security Pipeline

Every message written to storage passes through a three-stage cryptographic pipeline:

```
1. ENCRYPT plaintext --AES-GCM-256--> (ciphertext, nonce)
                                    [master key from data/master.key]

2. SIGN (msg_id + room + sender + timestamp + nonce + ciphertext)
                      --Ed25519--> signature
                                  [per-sender private key]

3. STORE MongoDB.upsert(_id=msg_id) <- deduplication enforced
           { sender, ciphertext(hex), nonce(hex), signature(hex), timestamp }
```

On `GET /feed`, the pipeline runs in reverse: the signature is verified and the ciphertext is decrypted. Tampered messages are flagged with `"tampered": true, "verified": false`.

2.4 Deduplication

Messages use MongoDB's `update_one(..., upsert=True)` with `_id = msg_id`. This guarantees that a message resubmitted by a retrying client appears only once in `/feed`, and that no duplicates appear across backends, since all three share the same MongoDB instance.

3 Load Balancer – Algorithm Deep Dive

The Load Balancer is written entirely in **Go** for concurrency and low overhead.

3.1 Core Data Structure

```
type BackendNode struct {
    URL *url.URL
    ActiveConns atomic.Int64 // live in-flight requests + WebSocket connections
    ewmaLatency float64 // Exponential Weighted Moving Average latency (ms)
    ewmaAlpha float64 // = 0.3 (30% weight to latest sample)
    Healthy bool
    Failures int
    Successes int
}
```

All counters use `sync/atomic`, so no mutex is needed for hot-path reads, keeping the balancer fast under concurrent load.

3.2 EWMA Latency Tracking

Rather than instantaneous latency (noisy) or a simple average (slow to react), the balancer tracks an Exponential Weighted Moving Average:

$$\text{EWMA}_{\text{new}} = \alpha \cdot \text{latency}_{\text{new}} + (1 - \alpha) \cdot \text{EWMA}_{\text{old}}, \qquad \alpha = 0.3$$

An α of 0.3 gives 30% weight to the newest measurement and 70% to history, smoothing transient spikes while still reacting to sustained load increases. Every HTTP response and every WebSocket handshake updates the EWMA of the backend that served it.

3.3 Performance Score Formula

Each backend is assigned a score (lower is better) on every routing decision:

```
score = (active_connections + 1) x EWMA_latency_ms

If EWMA_latency > threshold:
    penalty_ratio = EWMA_latency / threshold
    score *= (1.0 + penalty_ratio * 2.0)
```

Adding 1 to the active-connection count avoids a division by zero and stops an idle-but-slow backend from being blindly preferred over a busy-but-fast one. Once a backend's latency exceeds the threshold, its score is inflated super-linearly, and it only regains traffic once its EWMA drops back below the threshold.

3.4 Threshold-Aware Selection Algorithm

```
SelectBackend():
  1. Filter to healthy backends only
  2. For each healthy backend, compute score
  3. Separate into two groups:
       underThreshold: backends with EWMA <= threshold
       aboveThreshold: remaining healthy backends
```

```
  4. If underThreshold is non-empty -> return lowest-score from underThreshold
  5. Else (all backends loaded) -> return lowest-score overall
  6. Fallback: if all health checks failed -> route to all backends anyway
              (prevents complete outage during probe failure)
```

This is *not* round robin. Every request independently evaluates real-time scores, so a fast, idle backend receives far more traffic than a slow, busy one.

3.5 Active Health Checker

A background goroutine probes every backend every 10 seconds:

```
Probe: GET {backend}/feed?limit=1
  -> Success (HTTP 200-399): Successes++, Failures=0, EWMA updated with probe latency
  -> Failure (error / non-2xx): Failures++, Successes=0
  -> Marked UNHEALTHY after 2 consecutive failures
  -> Marked HEALTHY again after 2 consecutive successes (hysteresis)
```

Using `/feed?limit=1` as the probe is deliberate: it exercises the full stack (HTTP server *and* database read), not just TCP connectivity.

3.6 WebSocket Proxying

The Load Balancer performs a full WebSocket bridge, not a simple TCP tunnel:

```
Client --WS--> Load Balancer --WS--> Backend

  goroutine 1: Client -> Backend (frames forwarded, latency recorded)
  goroutine 2: Backend -> Client (frames forwarded, latency recorded)
  sync.Once done channel: either goroutine closing triggers full cleanup
```

The backend is selected once, at handshake time, and remains sticky for the lifetime of the connection, preserving message ordering and room-state consistency.

3.7 Supported Algorithms

```
--algorithm dynamic <- DEFAULT: EWMA + threshold + penalty scoring
--algorithm least_conn <- purely least active connections
--algorithm round_robin <- simple sequential rotation
```

3.8 Live Metrics Dashboard

The Load Balancer serves a live dashboard at `http://10.1.75.79:4201/lb/metrics`, reporting:

- **Per backend:** EWMA latency, active connections, performance score, health status, HTTP/WS/message counts, errors.
- **Global:** throughput (req/s), total HTTP requests, total WebSocket connections, active WebSocket sessions, total messages proxied.
- **Latency distributions:** P50 / P90 / P95 / P99 for HTTP responses, WebSocket handshakes, and message relay.
- Auto-refreshes every 2 seconds via `/lb/metrics.json`.

4 Benchmark Results

4.1 Load Sweep – `POST /message` and `GET /feed` via HTTP

Testing used the custom `load_generator.py` against the Load Balancer at `http://10.1.75.79:4201/`. Each client sent 8 messages and 2 feed requests.

Concurrent Clients	Throughput (req/s)	POST avg (ms)	POST P95 (ms)	Feed avg (ms)	Feed P95 (ms)	Errors
5	18.26	137.67	304.22	347.70	522.25	0
15	23.00	462.85	1,111.92	603.90	1,389.48	0
30	23.13	976.23	2,189.97	1,076.43	2,318.58	0
50	20.98	1,872.91	4,054.79	2,146.76	4,483.89	0

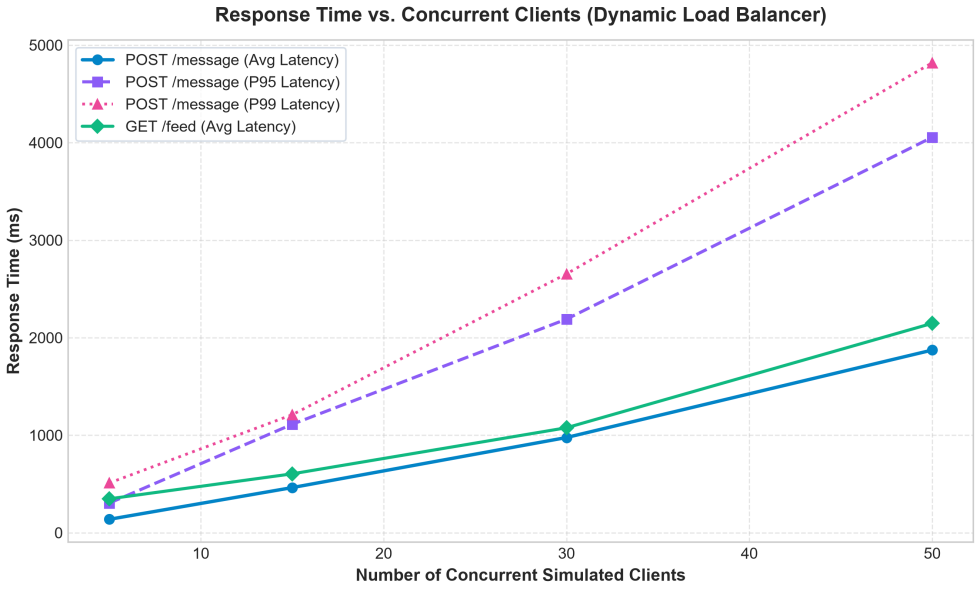

Figure 1: response time vs clients.

Key observations:

- Zero errors across all load levels; the Load Balancer and backends handled every request.
- Throughput peaks near 30 concurrent clients (≈23 req/s) and levels off at 50.
- P95 latency growth is expected: MongoDB Atlas adds roughly 100–200 ms per write, which compounds under high concurrency.

4.2 Threshold Calibration

The default threshold of **150 ms** was chosen from the benchmark sweep:

- At 5 clients, average latency is 137 ms, just under threshold, so no switching occurs.
- At 15+ clients, latency climbs above threshold, triggering penalty scoing and traffic redistribution across backends.
- This ensures the LB redistributes load only when a backend is actually degraded, not on routine traffic variation.

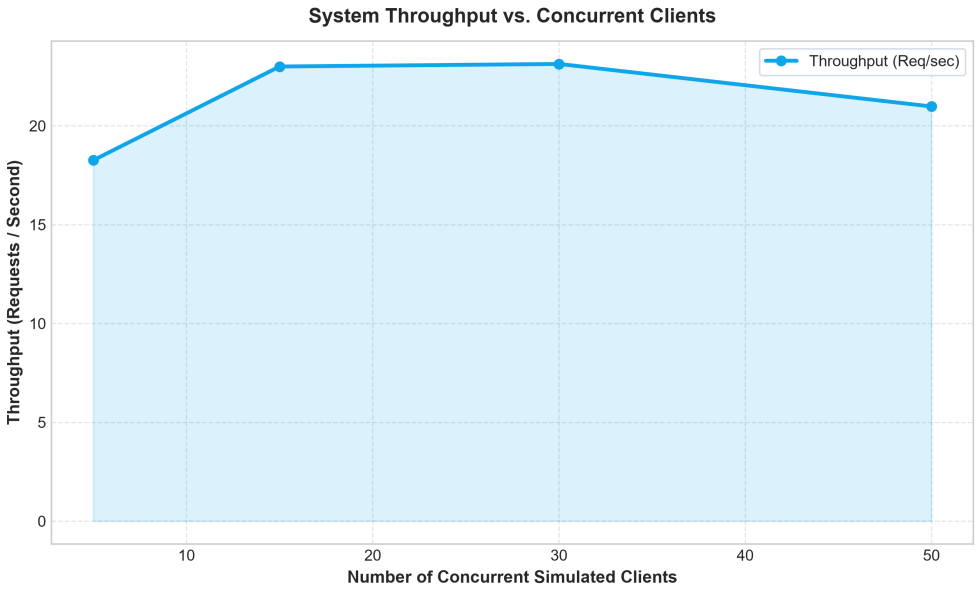

Figure 2: throughput vs clients.

5 Deployment

5.1 Backend (each of SYS1, SYS2, SYS3)

```
git clone <repo>
pip install -r requirements.txt
export PORT=3202
export MONGODB_URI="mongodb+srv://..." # shared Atlas URI
python app.py
```

5.2 Load Balancer

```
# Cross-compile on Windows for Linux target:
cmd /c "set GOOS=linux&& set GOARCH=amd64&& go build -o lb-linux main.go"
scp lb-linux student@<SYS_LB_IP>:~/go-lb/

# On the LB machine:
chmod +x lb-linux
./lb-linux \
  --port 4000 \
  --backends "http://10.1.75.79:3202,http://10.1.75.79:3202,http://10.1.75.79:3202" \
  --threshold 150.0 \
  --algorithm dynamic \
  --check-interval 10s
```

Load Balancer URL submitted: http://<10.1.75.79>:4000

5.3 Verifying the Required Routes

```
# POST /message
curl -X POST http://<10.1.75.79>:4000/message \
```

```
  -H "Content-Type: application/json" \
  -d '{"client-name": "alice", "msg": "hello world"}'

# GET /feed
curl http://<10.1.75.79>:4000/feed
```

6 Design Decisions and Trade-offs

Decision	Rationale
Go for the Load Balancer	Goroutines make concurrent WebSocket bridging trivial; `atomic` types give lock-free counters.
EWMA over simple average	Adapts to sustained load without overreacting to a single spike.
(active + 1) × EWMA score	Balances current load and historical speed in a single scalar.
Threshold + penalty	Avoids oscillation; backends only lose traffic after sustained degradation.
`/feed?limit=1` as health probe	Tests the full stack (HTTP + DB), not just a TCP ping.
2-failure / 2-success hysteresis	Prevents flapping; a backend must consistently fail or recover before its state changes.
MongoDB `upsert` deduplication	Ensures idempotent message storage across all three replicas.
AES-GCM + Ed25519 per message	Provides confidentiality (encryption) *and* integrity/authenticity (signature).
Shared MongoDB for all backends	Single source of truth: `/feed` returns consistent results regardless of which backend is hit.

7 File Index

File	Purpose
`app.py`	FastAPI app – `/message`, `/feed`, `/ws`, static files
`server/store.py`	MongoDB storage, AES-GCM encrypt/decrypt, Ed25519 sign/verify
`server/crypto.py`	Cryptographic primitives
`server/config.py`	Environment-based configuration
`load-balancer/go-lb/main.go`	Complete Go Load Balancer (837 lines)
`load-balancer/load_generator.py`	Load testing tool (HTTP and WebSocket modes)
`load-balancer/benchmark_and_plot.py`	Benchmark sweep and chart generator
`test_integration.py`	End-to-end API validation

8 LLM Usage Policy

Large Language Models (LLMs) were used as assistive tools during the development of this project in the following specific areas:

- **User Interface Design:** assisting in prototyping, structuring, and refining UI/UX components.
- **Backend Development:** providing syntax guidance, code optimization, and troubleshooting support for the Go implementation of the load balancer.